

Daily Paper Review: TRACE: Rollout-Guided Quantization-Aware Training for FP4 Reinforcement Learning of MoE Language Models

Auto-generated Report

October 8, 2026

1 Paper Summary

1.1 Problem and Motivation

Reinforcement learning for LLM post-training incurs substantial computation and memory overhead, particularly during rollout generation. For MoE models—which have massive total parameter counts but activate only a fraction per token (e.g., Qwen3.5-35B-A3B activates 3B of 35B)—this overhead is amplified by the need to load all expert weights. FP4 quantization during rollout can yield up to $5.4\times$ decoding speedup, but introduces a critical **train-rollout discrepancy problem**: the quantized rollout policy and the training-side policy diverge at FP4 rounding boundaries, causing policy mismatch, reward degradation, and entropy collapse.

Wang et al. (arXiv:2610.07767) show that FP4’s extremely coarse quantization grid (E2M1: 2 exponent bits, 1 mantissa bit) makes this problem qualitatively different from FP8. A concrete example: two activations differing by only 0.48 in BF16 can land on opposite sides of an FP4 rounding boundary, amplifying the discrepancy to 24 in the dequantized domain. For MoE models, the problem is compounded because small numerical differences can alter router scores and activate entirely different experts, causing cascading mismatches far more severe than in dense models.

1.2 Method

TRACE (Rollout-Guided Quantization-Aware Training) addresses the discrepancy at its root: rather than minimizing quantization error independently on each execution path, it uses the rollout-side FP4 rounding outcome to *guide* the training-side rounding decision.

Rollout-Guided Rounding. During rollout, the FP4 codeword q_{rollout} chosen at each quantization site is recorded. During the training forward pass, for each activation x at the corresponding site, the two nearest FP4 codewords (q_-, q_+) bracketing the normalized value $z = \text{clip}(x/s, -6, 6)$ are identified, and TRACE selects:

$$q_{\text{TRACE}} = \arg \min_{q \in \{q_-, q_+\}} |q - q_{\text{rollout}}| \quad (1)$$

This replaces standard round-to-nearest (RTN) with a rollout-aligned rounding rule. The formal guarantee is:

$$|q_{\text{TRACE}} - q_{\text{rollout}}| \leq |q_{\text{RTN}} - q_{\text{rollout}}| \quad (2)$$

ensuring TRACE *never increases* the train-rollout discrepancy compared to RTN. Gradients flow through via the straight-through estimator (STE).

Mantissa-Only Communication. Naively caching full rollout quantization information would require ~ 51 TB per training step for a typical configuration. TRACE exploits three observations for compression: (1) training and rollout share weight quantization scales, (2) only the low-order mantissa bits (plus the scale) suffice to reconstruct which FP4 codeword was chosen, and (3) deeper layers show more discrepancy amplification than shallow ones. The default configuration caches **1-bit mantissa from the latter 20 layers**, reducing storage to ~ 7.5 KB per generated token—a $\sim 6\times$ reduction over full caching. Ablations show 1-bit mantissa is nearly as effective as 4-bit (75.8 vs 75.9 average accuracy).

MoE-Specific Design. Only routed experts are quantized to FP4; non-expert modules (attention, shared experts, embeddings) remain in BF16. R3 routing replay ensures expert-routing decisions from rollout are replayed during training to prevent routing mismatch.

1.3 Results

Experiments span four Qwen-family MoE models from 35B to 2.4T total parameters across reasoning, coding, and long-horizon RL tasks.

On reasoning RL (Qwen3.5-35B-A3B, LiveCodeBench/AIME24/AIME25/HMMT25), TRACE achieves **75.3% average—exceeding the BF16 upper bound of 74.9%**—while all other FP4 methods show 6.1–16.3 point degradation (QAT: 59.6, QUADS: 68.8). At larger scales, TRACE matches BF16 within 0.4 points on DeepSWE (122B model), exceeds BF16 by 1.8 points on Terminal-Bench (125B), and matches within 0.1 on GDPval (2.4T model with 95B active parameters).

Efficiency gains are substantial: up to **$5.4\times$ decoding speedup** over BF16 at 128K output length, with only 7.4% end-to-end RL training overhead from guidance caching. Post-hoc quantization methods (Vanilla NVFP4, 4over6, H-Scale) achieve at best 71.4 average—3.9 points below TRACE—demonstrating that training with FP4 awareness is fundamentally superior to post-training quantization.

Training dynamics analysis reveals that vanilla QAT exhibits increasing train-rollout log-probability divergence leading to entropy collapse (0.40 $\rightarrow$ 0.26), while TRACE maintains stable alignment with entropy near the BF16 reference throughout training.

1.4 Limitations

The paper does not include an explicit limitations section. Implicit limitations include: (1) storage overhead of ~ 7.5 KB per token for guidance caching adds 7.4% end-to-end overhead; (2) TRACE reduces rounding-induced discrepancy but does not address activation differences from asynchronous policy updates; (3) all experiments use Qwen-family MoE models—generalization to dense models or other MoE architectures is not demonstrated; (4) layer selectivity (latter 20 of 40 layers) is empirically rather than theoretically motivated; (5) dependence on R3 routing replay for MoE routing consistency.

1.5 Relevance to Research

TRACE directly addresses LLM efficiency (Topic T1) at the intersection of quantization and RL training. The train-rollout discrepancy problem is a quantization-specific manifestation of the broader policy mismatch issue in RL, connecting to the SFT-vs-RL interference analysis of Review #110 (SFT Conflicts, RL Coexists), where gradient interference was characterized as norm-limited vs variance-limited. TRACE’s rollout-guided rounding operates at the activation level to prevent cascading MoE routing divergence, complementing FreeToken’s bandwidth-adaptive MoE serving (Review #117) which optimized inference-time expert delivery. The multi-reward RL training of SA-MRPO (Review #116) would directly benefit from TRACE’s

FP4 QAT, as saturation-aware advantage reweighting already addresses optimization-level challenges while TRACE solves the precision-level challenge.

2 Follow-up Research Directions

2.1 Direction 1: Rollout-Guided Quantization for Diffusion LLM Training

Gap: TRACE is designed for autoregressive GRPO where rollout and training share a clear sequential correspondence—each token position maps between the two paths. Diffusion LLMs generate tokens through iterative denoising, where the correspondence between rollout-side and training-side quantization sites is non-trivial: the same token position is visited multiple times across denoising steps with different activation distributions at each step.

Approach: Extend rollout-guided rounding to diffusion LLM RL training (e.g., V-GRPO from Review #42, Flow-DPPO from Review #69). Cache FP4 rounding decisions per denoising step and per position, creating a $\text{step} \times \text{position}$ guidance tensor. For the t -th denoising step at position p , use the corresponding rollout FP4 codeword to guide training-side rounding. This requires solving the step-correspondence problem when training and rollout use different noise schedules or number of function evaluations.

Feasibility: Medium. The step-correspondence problem is the main challenge—diffusion models may use different NFE counts during rollout vs training. Interpolation between cached guidance points may suffice when step counts differ.

2.2 Direction 2: FP4 Quantization-Aware Continual RL with Evolving Expert Pools

Gap: TRACE assumes a fixed set of MoE experts throughout training. In continual learning settings where new experts are dynamically added (as in Macaron-V1’s Mixture-of-LoRA, Review #111), the routing landscape evolves, and previously cached rollout guidance becomes stale for the new expert configuration. The train-rollout discrepancy may be amplified when FP4 rounding interacts with an evolving router that must learn to route to newly added experts.

Approach: Maintain a guidance staleness tracker that invalidates cached rounding decisions when the router distribution shifts beyond a threshold (measured via KL divergence between successive routing probability vectors). When guidance is invalidated for specific layers, fall back to standard RTN for those sites until fresh rollout guidance is available. Additionally, apply TRACE selectively—use full-precision rollout for the first few steps after expert pool expansion to establish stable routing, then switch to FP4 with guidance caching once routing stabilizes.

Feasibility: Medium-high. The staleness tracking adds minimal overhead, and the selective precision switching is a straightforward engineering extension. The main risk is that frequent expert additions may keep guidance perpetually stale.

2.3 Direction 3: Cross-Agent Rollout Guidance for Label-Free FP4 RL

Gap: TRACE caches rollout guidance from the same model being trained. In Co-RL’s cross-agent supervision framework (Review #118), where peer models provide rewards rather than ground-truth labels, the rollout guidance would need to come from a *different* model—but different models have different weight matrices, different quantization scales, and different activation distributions, making direct rollout guidance inapplicable.

Approach: Instead of caching FP4 codewords from the peer model (which have incompatible scales), cache the peer’s *routing decisions* and use these to stabilize the training model’s MoE routing under FP4 quantization. When the peer selects experts $\{e_1, e_3, e_7\}$ for a given token during its FP4 rollout, bias the training model’s router to select the same experts (via a

soft routing constraint), preventing FP4 rounding from causing routing divergence. This combines TRACE’s rounding alignment for within-model quantization with Co-RL’s cross-model supervision, enabling label-free FP4 RL at scale.

Feasibility: Medium. Cross-model routing alignment requires that the models share a compatible expert space (e.g., same number of experts), which holds for same-architecture pairs but not for cross-family pairs. The soft routing constraint must be carefully calibrated to avoid over-constraining the router.

3 Conclusion

TRACE solves a fundamental problem in FP4 RL training for MoE language models: the amplification of tiny activation differences across FP4 rounding boundaries into large policy mismatches, particularly through cascading MoE routing divergence. The solution—using cached rollout FP4 codewords to guide training-side rounding decisions—is both principled (provably non-increasing discrepancy) and practical (1-bit mantissa from 20 layers suffices, adding only 7.4% overhead). The result that FP4 TRACE *exceeds* BF16 accuracy while providing $5.4\times$ decoding speedup challenges the assumption that lower precision necessarily trades accuracy for efficiency. As RL training scales to trillion-parameter MoE models with increasingly long rollout horizons, TRACE’s approach of aligning quantization decisions between execution paths will be essential infrastructure for making frontier-scale RL economically viable.

References

- [1] X. Wang, H. Yu, Z. Zhuge, B. Mao, Z. Li, J. Feng, Y. Luo, Y. Zhang, Y. Cao, M. Zhang, D. Liu, and J. Zhang. TRACE: Rollout-Guided Quantization-Aware Training for FP4 Reinforcement Learning of MoE Language Models. *arXiv preprint arXiv:2610.07767*, 2026.
- [2] Z. Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024.
- [3] T. Dettmers et al. QLoRA: Efficient Finetuning of Quantized Language Models. In *Proc. NeurIPS*, 2023.
- [4] Q. Ma et al. R3: Rollout Routing Replay for MoE RL Training. *arXiv preprint*, 2025.
- [5] Y. Wang et al. Learn What’s Left, Not What’s Mastered: Saturation Aware Advantage Reweighting for Multi-Reward Policy Optimization. *arXiv preprint arXiv:2608.16072*, 2026.